

Project 5: Extending an Addition LLM to Subtraction and Reverse-Order Prediction

Group 4

Student 1: Karan Rooprai
Student 2: Nhu Hieu Nguyen

Student 1 ID: n12498122
Student 2 ID: n12194778

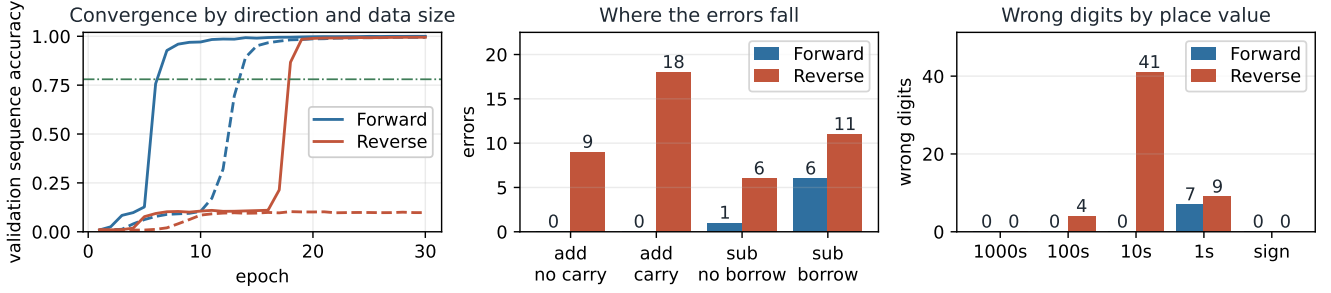


Figure 1: Left: validation sequence accuracy per epoch for both directions, at the full training set of 150,000 (solid) and at 80,000 examples (dashed); the dash-dot line marks the 0.78 target. **Centre and right:** error counts rather than accuracies, because every accuracy here exceeds 0.98 and rate bars would look identical. Forward makes 7 errors in 10,000 examples, Reverse 44, and they fall in different places.

The Week 7 workshop trains a small causal Transformer to add by next-token prediction, completing a prompt such as $23+1=$ with 24. Task 1 extends it to subtraction with signed results, Task 2 trains a second model that emits the answer digits right to left, and Task 3 compares them on one shared held-out set. The tokeniser, architecture, causal-masked objective and training loop follow the workshop; only the changes described below differ.

1. Method description

Task 1, handling subtraction. Two changes were needed. First, the vocabulary gains a single $-$ token with two roles: the subtraction operator in a prompt ($50-73=$) and the sign of a negative result (-23). Reusing one symbol keeps the vocabulary at fifteen tokens and lets the model infer from position which role is meant, which self-attention can represent and a second token would have hidden. Second, the generator samples an operation as well as two non-negative operands of at most three digits; Python’s `str` of a negative integer already produces the signed target, so no special case is needed when $b > a$. Training data is a balanced mix, half addition and half subtraction, with unique prompts and disjoint train, validation and test splits drawn by slicing one shuffled list. Three training settings differ, because borrows and signs make subtraction harder than addition: 30 epochs rather than the workshop’s 15, a cosine schedule annealing the learning rate to zero, and 150,000 training examples rather than 50,000. Batch size stays at 100.

Task 2, reverse-prediction implementation. Only the target is reversed; the prompt is untouched, so both models receive identical inputs and differ solely in output ordering. The answer string is reversed literally, so 31 becomes 13 and the model emits the units digit first, matching the direction in which carries and borrows propagate by hand. The brief does not define how to reverse a negative result, so the same literal rule is applied: -123 becomes $321-$, placing the sign last. The sign depends only on which operand is larger, a fact the prompt carries at every step, so emitting it last withholds nothing from the model. At evaluation the generated string is un-reversed before being parsed.

2. Results and analysis (Task 3)

Set-up. Both models are evaluated on one shared held-out set of 10,000 examples, balanced between addition and subtraction (subtraction fraction 0.49). The set is regenerated from the seed inside `main_report.ipynb`, checked against the shipped file, and shares no prompt with training or validation. Validation only tracks convergence; no model is selected on it. Decoding is greedy, so the numbers reproduce exactly on CPU or GPU.

Operation robustness. Both models clear the task’s 0.78 target: Forward reaches 0.9993 overall (95% CI 0.9986 to 0.9997) and Reverse 0.9956 (0.9941 to 0.9967). Split by operation, Forward reaches 1.0000 on addition over 5,076 examples and 0.9986 on subtraction, and Reverse 0.9947 and 0.9965 respectively (Table 1).

Digit-level performance. Accuracy is reported per place value over the answers that actually have that place, rather than over zero-padded answers. Only 2,490 of 10,000 answers reach the thousands column, so padding would score the other 7,510 as correct there and drive every high position to 1.000. Measured that way, Forward is exact at the thousands, hundreds and tens and scores 0.9993 at the units, while Reverse is exact at the thousands and scores 0.9996 at the hundreds, 0.9959 at the tens and 0.9991 at the units. Sign accuracy is 1.0000 for both models, so the sign-last convention of Task 2 costs nothing. Split by operation, no subtraction answer reaches the thousands column, because operands of at most three digits can only pass 999 by adding, so that cell is empty by construction. Forward is exact at every place on addition, and Reverse is weakest at the tens in both, 0.9951 on addition and 0.9967 on subtraction. Token-level digit accuracy is 0.9998 for Forward and 0.9987 for Reverse, and sequence accuracy reproduces the overall rates exactly.

Direction effects. The two models were scored on identical examples, so the comparison is paired and McNemar’s exact test applies. Forward is right where Reverse is wrong 44 times, Reverse right where Forward is wrong 7 times, and the two agree on 9,949 of 10,000 examples; $p = 1.212 \times 10^{-7}$. The accuracy difference is therefore real rather than sampling noise, though it is small in absolute terms.

Mode	Split	n	Errors	Accuracy	95% CI
Forward	overall	10,000	7	0.9993	0.9986 to 0.9997
Forward	addition	5,076	0	1.0000	0.9992 to 1.0000
Forward	subtraction	4,924	7	0.9986	0.9971 to 0.9993
Reverse	overall	10,000	44	0.9956	0.9941 to 0.9967
Reverse	addition	5,076	27	0.9947	0.9923 to 0.9963
Reverse	subtraction	4,924	17	0.9965	0.9945 to 0.9978

Table 1: Accuracy with the count behind it and a Wilson interval. Both directions clear 0.78 by a wide margin, so the error counts are what separate them.

Trained on the full set, both directions converge, reaching 0.95 validation sequence accuracy at epoch 8 and 19 respectively. The reverse curve is the more striking of the two: it stays below a quarter of its final accuracy through epoch 17, then passes 0.78 at epoch 18 and 0.95 at epoch 19 (Figure 1, left). A transition that abrupt is a barrier being crossed rather than capacity being filled, which would also explain why the reduced reverse run, given fewer updates in the same epochs, never crosses it. At 80,000 examples the two orderings finish at a final validation sequence accuracy of 0.9945 and 0.0979, against 0.9997 and 0.9949 on the full set. The two set sizes are not comparable in epochs, but in optimiser updates they agree: Forward passes 0.95 after 12,000 updates on either set. Reverse never reaches 0.95 on the reduced set, which is what a shorter run looks like as much as a smaller one.

Error patterns. Each direction fails in a different region of the problem (Table 2).

All 7 of Forward’s errors fall on answers of a single digit, a group holding only 92 of the 10,000 examples (Figure 2, top), and every one is wrong by exactly one unit. Predicting left to right requires committing to the answer’s length before emitting any digit, so a three-digit subtraction that collapses to a single digit is the case that ordering makes hardest.

Of Reverse’s 44 errors, 42 fall on the 92 prompts whose first operand is 2 digits longer than the second, a rate of 0.4565 there against 2 errors across the other 9,908 examples. Among the errors that keep the right number of digits, 38 are wrong at the tens digit, the first place the model reaches after the shorter operand has run out of digits; 3 predictions have the wrong length instead. Emitting units first makes the units column trivial, because both operands end there, but it forces the model to detect that one operand is exhausted and carry the remainder alone. That is the alignment reverse ordering makes hardest, and it is the complement of Forward’s weakness.

Carrying and borrowing separate the two directions. Forward is exact on both addition cases, the 4,226 sums that carry somewhere and the 850 that carry nowhere, so all 7 of its errors are subtractions and 6 of those need a borrow. Reverse errs in all four cases, 27 on addition and 17 on subtraction. One split reads like a counterexample. Reverse is less accurate on additions that carry nowhere (0.9894 over 850 examples) than on additions that do carry (0.9957 over 4,226), which inverts the expectation that carrying is the harder case. Operand width resolves it: an addition that carries in no column tends to have a small second operand, so the no-carry group is enriched in the width mismatch above. Carrying is not what this model finds difficult; keeping the columns aligned is.

3. Strengths, limitations and recommendations

The comparison’s strength is its design. Both models share a tokeniser, architecture, training data, optimiser

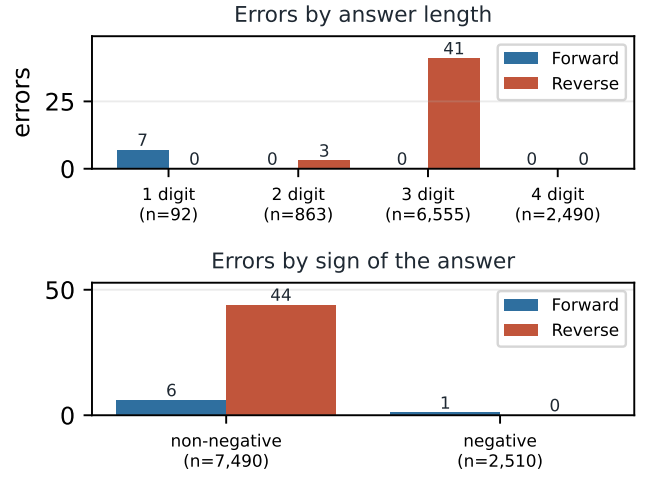


Figure 2: Top: errors by the number of digits in the answer, with the size of each group underneath. **Bottom:** errors by the sign of the answer, which the reverse ordering emits last. Both panels count the errors of Figure 1, cut by the answer’s shape rather than by operation.

and schedule, so prediction direction is the only variable between them, and both are scored on the same 10,000 examples, which makes the accuracy difference a paired comparison with an exact test behind it. Several limitations bound what it supports. Each model was trained once, from one seed, so the difference in convergence speed between the directions is a single observation rather than a distribution. Operands are capped at three digits, so nothing here shows whether either direction generalises to longer arithmetic, which is where the error analysis suggests they would diverge. Positions are encoded absolutely, following the workshop, so whether the encoding causes Reverse’s alignment errors is untested. The ablation also held epochs fixed instead of optimiser updates, so fixing updates would test data volume more directly.

The recommendations follow from those. Train several seeds before treating the convergence gap as a property of the ordering rather than of one optimisation trajectory. Extend the operand range and re-measure, because the failure modes found here are both about structure, answer length for Forward and operand width for Reverse, and both should grow with digits. Test a relative or learned positional encoding against the absolute one, since the reverse model’s errors cluster at exactly the place where relative positions would carry the alignment information it appears to lack. Finally, if one direction has to be chosen, choose Forward: it is more accurate on this test set by a margin the paired test separates from noise, it converges in fewer epochs, and its hardest case costs it 7 errors where Reverse’s costs 42.

Prompt	True	Predicted	Failure
<i>Forward: short answer, long operands</i>			
505-496	9	8	single digit slip
487-479	8	7	single digit slip
199-200	-1	-2	single digit slip
907-899	8	7	single digit slip
<i>Reverse: operands of very different width</i>			
309-0	309	319	single digit slip
195+2	197	187	single digit slip
809+1	810	800	single digit slip
730-8	722	733	2 digit slips

Table 2: Representative failures of each model, from the full error list in `main_report.ipynb`. The two sets have no case in common.